YOLOv11 Garbage Recognition

1. YOLOv11 Introduction

YOLOv11 is a **lightweight, high-precision, real-time object detection model** launched by Ultralytics at the end of 2024. It achieves a better balance in speed, accuracy, and parameter count, making it suitable for edge and industrial deployment. It focuses on the optimal solution for **speed - accuracy - deployment cost**, while maintaining YOLOv8 ecosystem compatibility, with fewer parameters, faster inference, and higher accuracy. It supports **detection, segmentation, pose estimation, OBB, classification, tracking**. Ultralytics YOLO is compatible with ONNX/TensorRT/TFLite, making it easy to deploy to edge devices (Jetson, CPU).

Core advantages include the following:

- **Lighter and Faster**: Reduced parameter count, increased speed (CPU inference about 30% faster).
- **Higher Accuracy**: **2–5% mAP** improvement over YOLOv8.
- **Stronger Small Object Detection**: Attention mechanism optimization, significant improvement in small object AP.
- **Deployment Friendly**: Compatible with Ultralytics ecosystem, convenient quantization and export.

Main use scenarios:

- Real-time detection on edge devices (security, robotics, autonomous driving)
- Industrial quality inspection, medical imaging, aerospace remote sensing
- Mobile / embedded vision applications

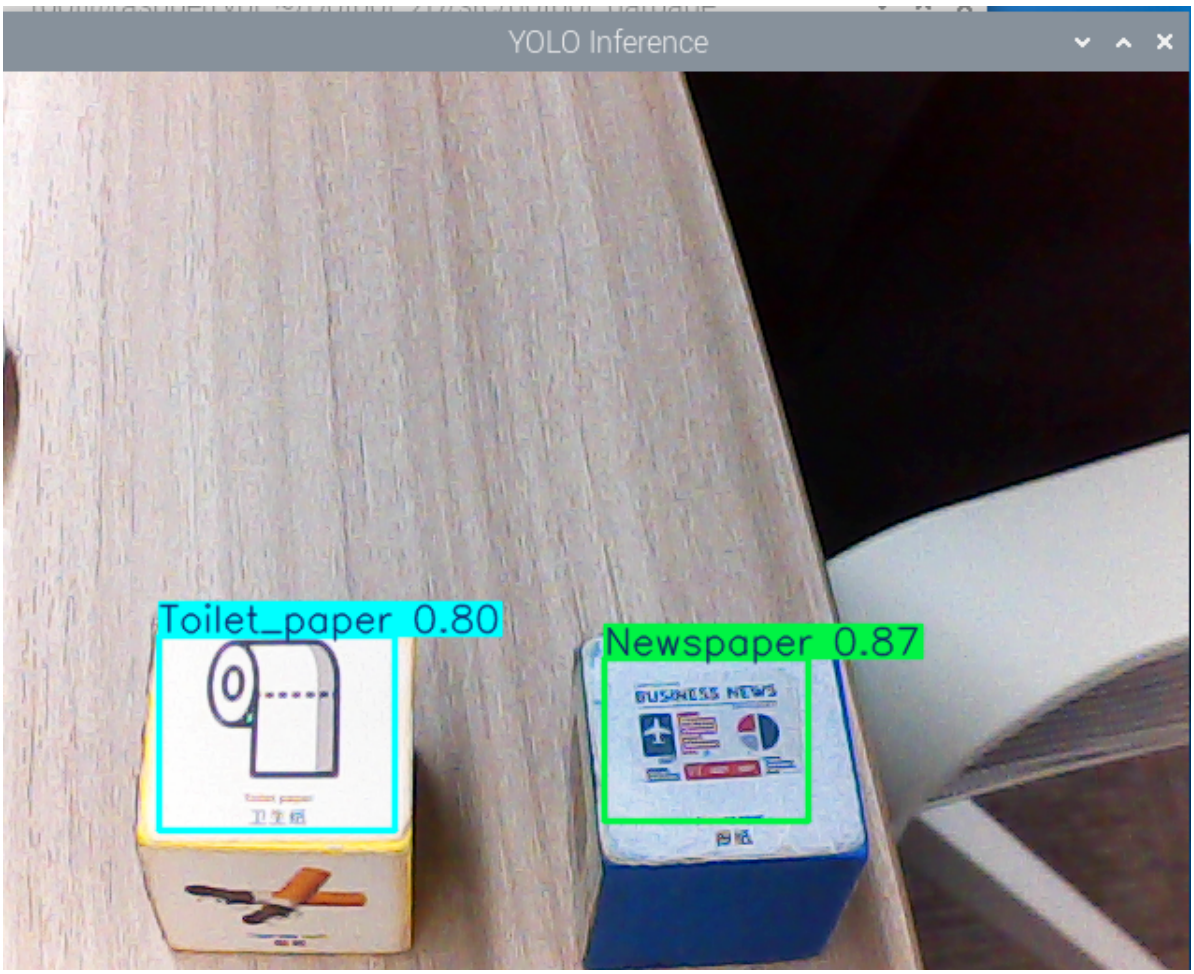
2. Function Startup

After the program starts, it will load the trained model and recognize garbage on the garbage label code blocks. After entering the main board desktop, open the terminal and run the following command to start the corresponding Dofbot container:

```
./Docker_Ros.sh
```

Then enter the following command in the docker terminal to start,

```
python3 /root/Dofbot_2D/src/dofbot_garbage/garbage_detect.py
```



Press the q key to exit the program.

Program code path: /root/Dofbot_2D/src/dofbot_garbage/garbage_detect.py

```
import cv2
from ultralytics import YOLO
from Arm_Lib import Arm_Device
Arm = Arm_Device()
init_joints = [90.0, 120.0, 0.0, 0.0, 90.0, 0.0]
Arm.Arm_serial_servo_write6_array(init_joints,2000)

# Load the YOLO model
model = YOLO("/root/ultralytics/ultralytics/best.onnx")

# Open the camera
cap = cv2.VideoCapture(0)
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
# Get the video frame size and frame rate
frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
# Define the codec and create a VideoWriter object to output the processed video
# Loop through the video frames
while cap.isOpened():
    # Read a frame from the video
    success, frame = cap.read()
    if success:
        # Run YOLO inference on the frame
        results = model(frame)
        boxes = results[0].boxes
```

```
        if boxes != [None]:
            for box in boxes:
                class_id = int(box.cls)
                label = model.names[class_id]
                print("label: ",label)
        # Visualize the results on the frame
        annotated_frame = results[0].plot()
        # Display the annotated frame
        cv2.imshow("YOLO Inference", cv2.resize(annotated_frame, (640, 480)))

        # Break the loop if 'q' is pressed
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break
    else:
        # Break the loop if the end of the video is reached
        break
# Release the video capture and writer objects, and close the display window
cap.release()
cv2.destroyAllWindows()
```